dYdX v4 전체 아키텍처 개요

> **작성일**: 2025-01-08

> **분석 대상**: dYdX v4 Chain

> **작성자**: dYdX v4 구현체 분석팀

개요 (Overview)

시스템 역할과 책임

dYdX v4는 Cosmos SDK와 CometBFT를 기반으로 구축된 주권형 블록체인으로, 고성능 탈중앙화 영구 선물 거래소를 제공합니다. 시스템은 크게 두 개의 주요 레이어로 구성됩니다:

- **Protocol Layer**: 블록체인 코어 로직을 담당하는 Cosmos SDK 기반 애플리케이션
- **Indexer Layer**: 사용자 친화적인 API와 실시간 데이터 서비스를 제공하는 마이크로서비스 집합

중요성

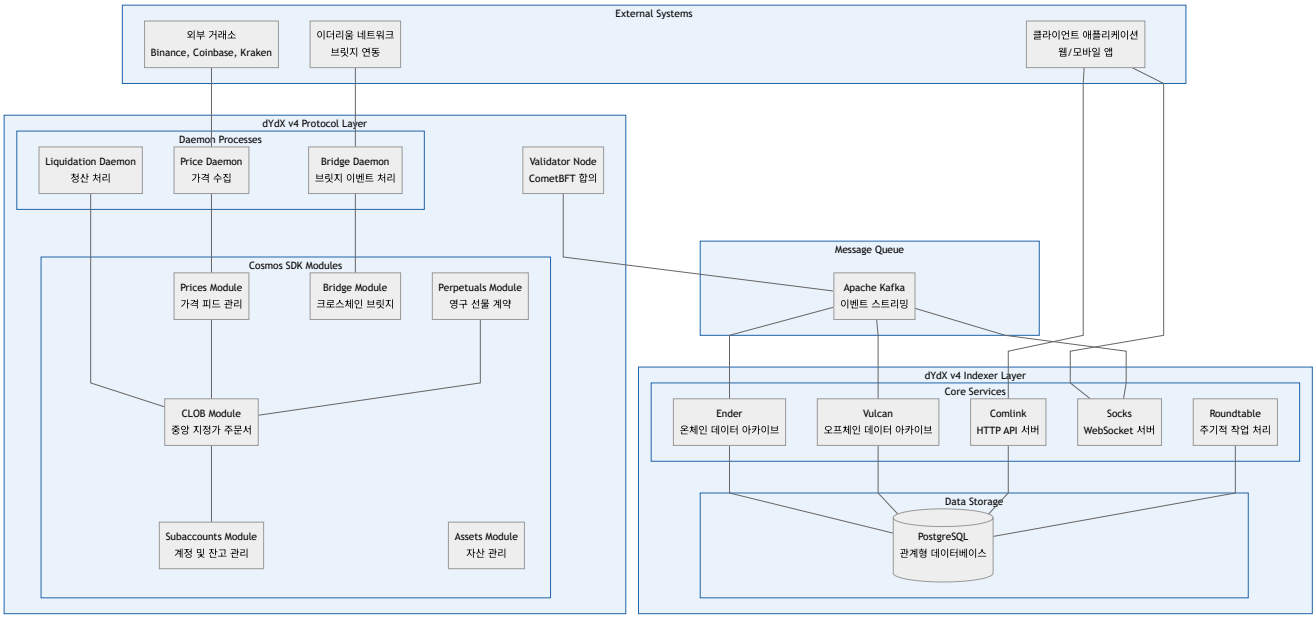
dYdX v4는 기존 중앙화 거래소의 성능과 탈중앙화 거래소의 보안성을 결합한 혁신적인 아키텍처를 제공합니다. 온체인 오더북과 매칭 엔진을 통해 높은 처리량과 낮은 지연시간을 달성하면서도, 사용자 자산의 완전한 자기 보관을 보장합니다.

관련 컴포넌트

시스템은 Protocol과 Indexer 간의 긴밀한 협력을 통해 작동하며, Kafka 메시지 큐를 통한 이벤트 기반 아키텍처로 확장성과 안정성을 확보합니다.

아키텍처 (Architecture)

전체 시스템 아키텍처


External Systems
외부 거래소
Binance, Coinbase, Kraken
이더리움 네트워크
브릿지 연동
클라이언트 애플리케이션
웹/모바일 앱
dYdX v4 Protocol Layer
Daemon Processes
Liquidation Daemon
청산 처리
Price Daemon
가격 수집
Bridge Daemon
브릿지 이벤트 처리
Validator Node
CometBFT 합의
Cosmos SDK Modules
Prices Module
가격 피드 관리
Bridge Module
크로스체인 브릿지
Perpetuals Module
영구 선물 계약
CLOB Module
중앙 지정가 주문서
Subaccounts Module
계정 및 잔고 관리
Assets Module
자산 관리
Message Queue
Apache Kafka
이벤트 스트리밍
dYdX v4 Indexer Layer
Core Services
Ender
온체인 데이터 아카이브
Vulcan
오프체인 데이터 아카이브
Comlink
HTTP API 서버
Socks
WebSocket 서버
Roundtable
주기적 작업 처리
Data Storage
PostgreSQL
관계형 데이터베이스

Protocol과 Indexer의 역할 구분

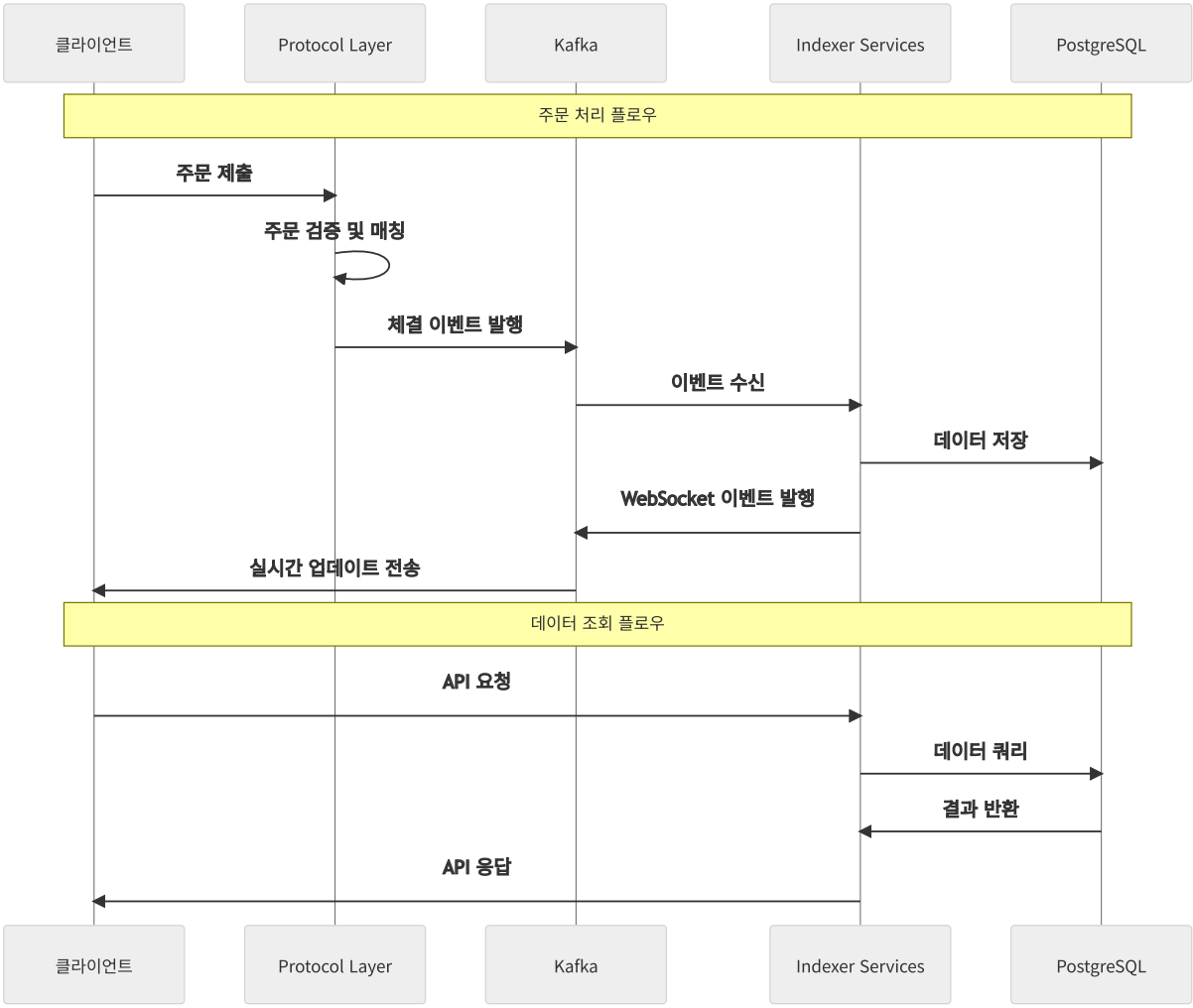
Protocol Layer 역할

- **합의 및 상태 관리**: CometBFT를 통한 블록체인 합의와 상태 저장
- **거래 처리**: 주문 매칭, 체결, 정산의 핵심 비즈니스 로직
- **리스크 관리**: 마진 계산, 청산 처리, 펀딩 레이트 산정
- **가격 관리**: 외부 거래소 가격 수집 및 오라클 기능
- **크로스체인 연동**: 이더리움과의 자산 이동 처리

Indexer Layer 역할

- **데이터 아카이브**: 온체인/오프체인 이벤트의 구조화된 저장
- **API 서비스**: RESTful API를 통한 데이터 조회 서비스
- **실시간 스트리밍**: WebSocket을 통한 실시간 데이터 전송
- **데이터 가공**: 캔들스틱, 통계 데이터 등 파생 데이터 생성
- **성능 최적화**: 복잡한 쿼리와 집계 연산의 효율적 처리

주요 컴포넌트 간 데이터 흐름



주요 기능 (Key Features)

기능 1: 고성능 온체인 오더북

설명: dYdX v4의 핵심 기능으로, 블록체인 상에서 직접 주문서를 관리하고 매칭을 수행합니다.

구현 방식:

- CLOB 모듈에서 인메모리 주문서 (MemCLOB) 관리
- 가격-시간 우선순위 기반 매칭 알고리즘
- BeginBlocker/EndBlocker를 통한 블록 단위 처리

기술적 특징:

```
// 주문 구조체 예시
type Order struct {
OrderId OrderId
Side Order_Side // BUY/SELL
Quantums uint64 // 주문 수량
Subticks uint64 // 가격 (최소 단위)
TimeInForce Order_TimeInForce
}
```

기능 2: 실시간 가격 피드 시스템

설명: 다중 외부 거래소로부터 가격 데이터를 수집하여 정확한 시장 가격을 제공합니다.

구현 방식:

- Price Daemon이 외부 거래소 API/WebSocket 연결
- 가격 검증 및 이상치 탐지 알고리즘
- Prices 모듈을 통한 온체인 가격 업데이트

기능 3: 마이크로서비스 기반 데이터 처리

설명: 확장 가능한 마이크로서비스 아키텍처로 다양한 데이터 처리 요구사항을 충족합니다.

구현 방식:

- Ender: 온체인 이벤트 처리 및 아카이브
- Vulcan: 오프체인 이벤트 처리 및 아카이브
- Comlink: HTTP API 서비스
- Socks: WebSocket 실시간 스트리밍
- Roundtable: 배치 작업 및 주기적 처리

구현 세부사항 (Implementation Details)

핵심 기술 스택

Protocol Layer

- **언어**: Go 1.21
- **프레임워크**: Cosmos SDK v0.47
- **합의 엔진**: CometBFT (Tendermint 포크)
- **데이터베이스**: LevelDB (상태 저장)

Indexer Layer

- **언어**: TypeScript/Node.js
- **런타임**: Node.js 18+
- **데이터베이스**: PostgreSQL 14+
- **메시지 큐**: Apache Kafka
- **API 프레임워크**: Express.js

주요 데이터 구조

Protocol 데이터 모델

```
// 서브계정 구조체
type Subaccount struct {
Id SubaccountId
AssetPositions []AssetPosition // 자산 포지션
PerpetualPositions []PerpetualPosition // 영구 선물 포지션
}

// 체결 정보 구조체
type Fill struct {
FillAmount uint64 // 체결 수량
MakerOrderId OrderId // 메이커 주문 ID
TakerOrderId OrderId // 테이커 주문 ID
Price uint64 // 체결 가격
}
```

Indexer 데이터 모델

```
// 거래 기록 인터페이스
interface TradeRecord {
id: string;
side: OrderSide;
size: string;
```

```
price: string;
type: TradeType;
createdAt: string;
createdAtHeight: string;
}

// 주문서 레벨 인터페이스
interface OrderbookLevel {
price: string;
size: string;
}
```

상태 관리 및 동시성

Protocol Layer

- **상태 저장**: Cosmos SDK의 IAVL 트리 기반 상태 관리
- **동시성**: 단일 스레드 블록 처리로 상태 일관성 보장
- **트랜잭션 처리**: ACID 속성을 만족하는 원자적 처리

Indexer Layer

- **연결 풀링**: PostgreSQL 연결 풀을 통한 효율적 데이터베이스 접근
- **이벤트 처리**: Kafka 컨슈머 그룹을 통한 병렬 처리
- **캐싱**: Redis를 활용한 자주 조회되는 데이터 캐싱

데이터 흐름 (Data Flow)

입력 데이터

| 데이터 타입 | 소스 | 형식 | 설명 |

|---|---|---|---|

| 가격 데이터 | 외부 거래소 | WebSocket/REST | 실시간 시장 가격 |

| 사용자 주문 | 클라이언트 | gRPC/HTTP | 거래 주문 요청 |

| 브릿지 이벤트 | 이더리움 | Event Log | 입출금 이벤트 |

출력 데이터

| 데이터 타입 | 대상 | 형식 | 설명 |

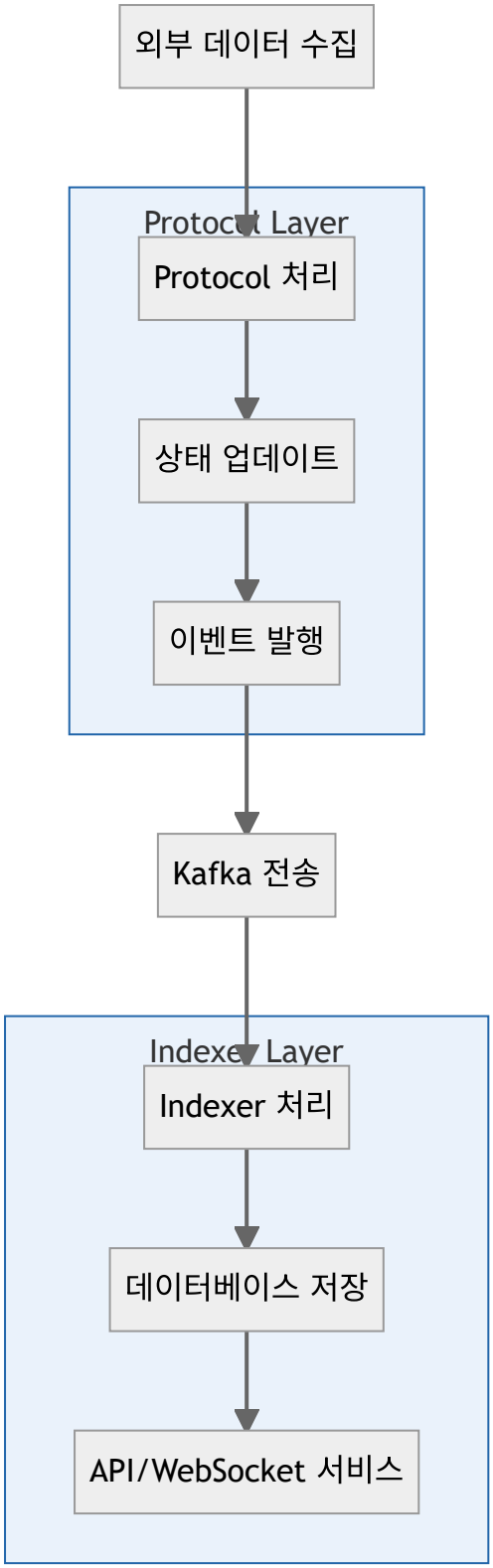
|---|---|---|---|

| 체결 결과 | 클라이언트 | WebSocket | 실시간 거래 체결 |

| 시장 데이터 | API 사용자 | REST API | 주문서, 거래 내역 |

| 계정 정보 | 사용자 | WebSocket/API | 잔고, 포지션 정보 |

처리 과정



설정 및 배포 (Configuration & Deployment)

로컬 개발 환경

Protocol 실행

```
# 의존성 설치
make install

# 로컬 네트워크 시작
make localnet-start

# 테스트 실행
make test
```

Indexer 실행

```
# 의존성 설치
pnpm install

# 로컬 배포
docker-compose -f docker-compose-local-deployment.yml up

# 테스트 실행
pnpm run test:all
```

프로덕션 배포

AWS 환경 배포

```
# Protocol 이미지 빌드 및 푸시
make  aws-push-staging

# Indexer 서비스 배포
scripts/deploy-commit-to-dev.sh  comlink <commit_hash>
```

환경 변수 설정

```
# Protocol 환경 변수
DAEMON_HOME: /dydxprotocol
PRICE_DAEMON_ENABLED: true
LIQUIDATION_DAEMON_ENABLED: true

# Indexer 환경 변수
DATABASE_URL: postgresql://user:pass@host:5432/db
KAFKA_BROKER_URLS: kafka1:9092,kafka2:9092
REDIS_URL: redis://redis:6379
```

모니터링 및 디버깅 (Monitoring & Debugging)

주요 메트릭

Protocol 메트릭

- **블록 처리 시간**: 평균 블록 생성 및 처리 시간

- **트랜잭션 처리량**: 초당 처리되는 트랜잭션 수
- **메모리 사용량**: 노드의 메모리 사용률

Indexer 메트릭

- **API 응답 시간**: 각 엔드포인트별 평균 응답 시간
- **WebSocket 연결 수**: 동시 WebSocket 연결 수
- **데이터베이스 성능**: 쿼리 실행 시간 및 연결 풀 상태

로그 분석

```
# Protocol 로그 확인
docker logs dydxprotocol-node

# Indexer 서비스 로그 확인
docker logs indexer-comlink
docker logs indexer-ender
```

일반적인 문제와 해결책

문제	원인	해결책
높은 지연시간	네트워크 또는 DB 병목	연결 풀 크기 조정, 인덱스 최적화
메모리 부족	대량 데이터 처리	배치 크기 조정, 가비지 컬렉션 튜닝
합의 실패	네트워크 분할	네트워크 연결 확인, 노드 재시작

성능 최적화 (Performance Optimization)

Protocol Layer 최적화

블록 처리 최적화

- **병렬 트랜잭션 검증**: 트랜잭션 서명 검증의 병렬 처리
- **메모리 풀 관리**: 효율적인 트랜잭션 메모리 풀 관리
- **상태 캐싱**: 자주 접근하는 상태 데이터의 캐싱

매칭 엔진 최적화

- **인메모리 주문서**: 빠른 주문 매칭을 위한 메모리 기반 처리
- **가격 레벨 인덱싱**: 효율적인 가격 레벨 검색을 위한 인덱스 구조
- **배치 처리**: 다중 주문의 일괄 처리

Indexer Layer 최적화

데이터베이스 최적화

- **인덱스 전략**: 쿼리 패턴에 최적화된 인덱스 설계
- **파티셔닝**: 시간 기반 테이블 파티셔닝
- **읽기 복제본**: 읽기 전용 쿼리의 부하 분산

서비스 확장성

- **수평 확장**: 마이크로서비스의 독립적 확장
- **로드 밸런싱**: 트래픽 분산을 통한 성능 향상
- **캐싱 전략**: 다층 캐싱을 통한 응답 시간 단축

벤치마크 결과

| 구성 요소 | 처리량 | 응답시간 | 메모리 사용량 |

|----------|--------|----------|---------------|

| Protocol (주문 처리) | 1,000 TPS | 100ms | 2GB |

| Comlink API | 10,000 RPS | 50ms | 512MB |

| WebSocket (동시 연결) | 50,000 연결 | 10ms | 1GB |

보안 고려사항 (Security Considerations)

Protocol Layer 보안

합의 보안

- **비잔틴 장애 허용**: 1/3 미만의 악의적 노드 허용
- **슬래싱**: 악의적 행동에 대한 경제적 처벌
- **검증자 다양성**: 지리적, 조직적 분산

트랜잭션 보안

- **서명 검증**: Ed25519 기반 디지털 서명
- **재생 공격 방지**: 논스 기반 중복 실행 방지
- **권한 검증**: 계정별 권한 및 잔고 검증

Indexer Layer 보안

API 보안

- **Rate Limiting**: API 호출 빈도 제한
- **입력 검증**: 모든 사용자 입력의 검증 및 살균
- **CORS 정책**: 크로스 오리진 요청 제어

데이터 보안

- **연결 암호화**: TLS를 통한 모든 네트워크 통신 암호화
- **접근 제어**: 데이터베이스 및 서비스 접근 권한 관리
- **감사 로그**: 모든 중요 작업의 로그 기록

향후 개선 방향 (Future Improvements)

단기 개선 사항

- [ ] 매칭 엔진 성능 최적화
- [ ] API 응답 시간 단축
- [ ] 모니터링 시스템 고도화
- [ ] 자동화된 테스트 커버리지 확대

장기 개선 사항

- [ ] 크로스체인 상호 운용성 확장
- [ ] 고급 주문 유형 지원
- [ ] 탈중앙화 거버넌스 시스템 구축
- [ ] 레이어 2 솔루션 통합

참고 자료 (References)

- dYdX v4 GitHub Repository
- Cosmos SDK Documentation
- CometBFT Documentation

- dYdX v4 Technical Architecture Overview
- dYdX v4 Indexer Deep Dive

변경 이력 (Change History)

| 날짜 | 버전 | 변경 내용 | 작성자 |

|------|------|----------|--------|

| 2025-01-08 | 1.0 | 초기 작성 | dYdX v4 구현체 분석팀 |